

1. Kod źródłowy:

```
2. import os
3. import sys
4. from datetime import datetime
5. from flask import Flask, request, render_template_string
6. import requests
7.
8. app = Flask(__name__)
9.
10. # Konfiguracja danych autora i portu
11. AUTHOR = "Imię Nazwisko" # Wpisz tutaj swoje dane
12. PORT = int(os.environ.get("PORT", 8080))
13.
14. # Predefiniowana lista lokalizacji wraz ze współrzędnymi geograficznymi
    (szerokość, długość)
15. LOCATIONS = {
16.     "Polska": {
17.         "Warszawa": (52.2297, 21.0122),
18.         "Kraków": (50.0647, 19.9450)
19.     },
20.     "Hiszpania": {
21.         "Madryt": (40.4168, -3.7038),
22.         "Barcelona": (41.3874, 2.1686)
23.     }
24. }
25.
26. # Szablon HTML dla interfejsu użytkownika
27. HTML_TEMPLATE = """
28. <!DOCTYPE html>
29. <html>
30. <head><title>Aplikacja Pogodowa</title></head>
31. <body>
32.     <h2>Sprawdź pogodę</h2>
33.     <form method="POST">
34.         <label>Kraj:</label>
35.         <select name="country">
36.             {% for country in locations.keys() %}
37.                 <option value="{{ country }}">{{ country }}</option>
38.             {% endfor %}
39.         </select>
40.         <br><br>
41.         <label>Miasto:</label>
42.         <select name="city">
43.             <option value="Warszawa">Warszawa</option>
44.             <option value="Kraków">Kraków</option>
45.             <option value="Madryt">Madryt</option>
46.             <option value="Barcelona">Barcelona</option>
47.         </select>
```

```

48.         <br><br>
49.         <input type="submit" value="Sprawdź">
50.     </form>
51.
52.     {% if weather_data %}
53.         <h3>Pogoda dla: {{ selected_city }}, {{ selected_country }}</h3>
54.         <p>Temperatura: {{ weather_data['temperature'] }} °C</p>
55.         <p>Prędkość wiatru: {{ weather_data['windspeed'] }} km/h</p>
56.     {% endif %}
57. </body>
58. </html>
59. """
60.
61. # Funkcja uruchamiana przed pierwszym żądaniem
62. def log_startup_info():
63.     """Pozostawia w logach informację o dacie uruchomienia, autorze i
        porcie TCP"""
64.     startup_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
65.     # Używamy flush=True, aby upewnić się, że logi natychmiast trafiają do
        standardowego wyjścia w Dockerze
66.     print("=====", flush=True)
67.     print(f"Data uruchomienia kontenera: {startup_time}", flush=True)
68.     print(f"Autor programu: {AUTHOR}", flush=True)
69.     print(f"Aplikacja nasłuchuje na porcie TCP: {PORT}", flush=True)
70.     print("=====", flush=True)
71.
72. log_startup_info()
73.
74. @app.route("/", methods=["GET", "POST"])
75. def index():
76.     """Główny endpoint obsługujący formularz wyboru kraju/miasta i
        odpytujący API pogodowe"""
77.     weather_data = None
78.     selected_country = None
79.     selected_city = None
80.
81.     if request.method == "POST":
82.         selected_country = request.form.get("country")
83.         selected_city = request.form.get("city")
84.
85.         # Weryfikacja, czy wybrane miasto pasuje do predefiniowanej
        listy w danym kraju
86.         if selected_country in LOCATIONS and selected_city in
            LOCATIONS[selected_country]:
87.             lat, lon = LOCATIONS[selected_country][selected_city]
88.
89.             # Pobieranie aktualnej pogody z darmowego Open-Meteo API

```

```

90.         api_url = f"https://api.open-
meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"
91.         response = requests.get(api_url)
92.
93.         if response.status_code == 200:
94.             weather_data = response.json().get("current_weather")
95.
96.         return render_template_string(
97.             HTML_TEMPLATE,
98.             locations=LOCATIONS,
99.             weather_data=weather_data,
100.             selected_country=selected_country,
101.             selected_city=selected_city
102.         )
103.
104.     if __name__ == "__main__":
105.         # Uruchomienie serwera aplikacji
106.         app.run(host="0.0.0.0", port=PORT)

```

Sprawdź pogodę

Kraj:

Miasto:

Pogoda dla: Madryt, Hiszpania

Temperatura: 13.6 °C

Prędkość wiatru: 4.2 km/h

2. Dockerfile

ETAP 1: Builder

Wykorzystujemy lekki obraz Alpine do przygotowania zależności

FROM python:3.11-alpine AS builder

Ustawienie katalogu roboczego

WORKDIR /app

Kopiowanie wyłącznie pliku wymagań - optymalizacja cache'a (warstwa przebuduje się tylko gdy zmienimy wymagania)

COPY requirements.txt .

Instalacja zależności do przestrzeni użytkownika, aby łatwo je skopiować w kolejnym etapie

RUN pip install --user --no-cache-dir -r requirements.txt

ETAP 2: Obraz Docelowy (Runner)

Wykorzystujemy ten sam lekki obraz bazy

FROM python:3.11-alpine

Deklaracja autora zgodna ze standardem OCI

LABEL org.opencontainers.image.authors="Imię Nazwisko"

Ustawienie katalogu roboczego

WORKDIR /app

Kopiowanie zainstalowanych pakietów z etapu buildera (wieloetapowe budowanie - oszczędność miejsca)

COPY --from=builder /root/.local /root/.local

Kopiowanie kodu aplikacji

COPY app.py .

Zapewnienie, że skrypty zainstalowane przez pip są widoczne w zmiennej PATH

ENV PATH=/root/.local/bin:\$PATH

ENV PORT=8080

Informacja dla Dockera o nasłuchującym porcie

EXPOSE \$PORT

Konfiguracja instrukcji healthcheck - weryfikuje czy aplikacja prawidłowo odpowiada

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \

CMD wget --quiet --tries=1 --spider http://localhost:\$PORT/ || exit 1

Uruchomienie aplikacji

CMD ["python", "app.py"]

3.

a) docker build -t app-pogoda:latest .

b) docker run -d --name kontener-pogoda -p 8080:8080 app-pogoda:latest

c) docker logs kontener-pogoda

d) docker history app-pogoda:latest

docker images app-pogoda:latest